

Lab 6
Phina Penteriani
2364350

Edge impulse link: <https://studio.edgeimpulse.com/public/997976/live>

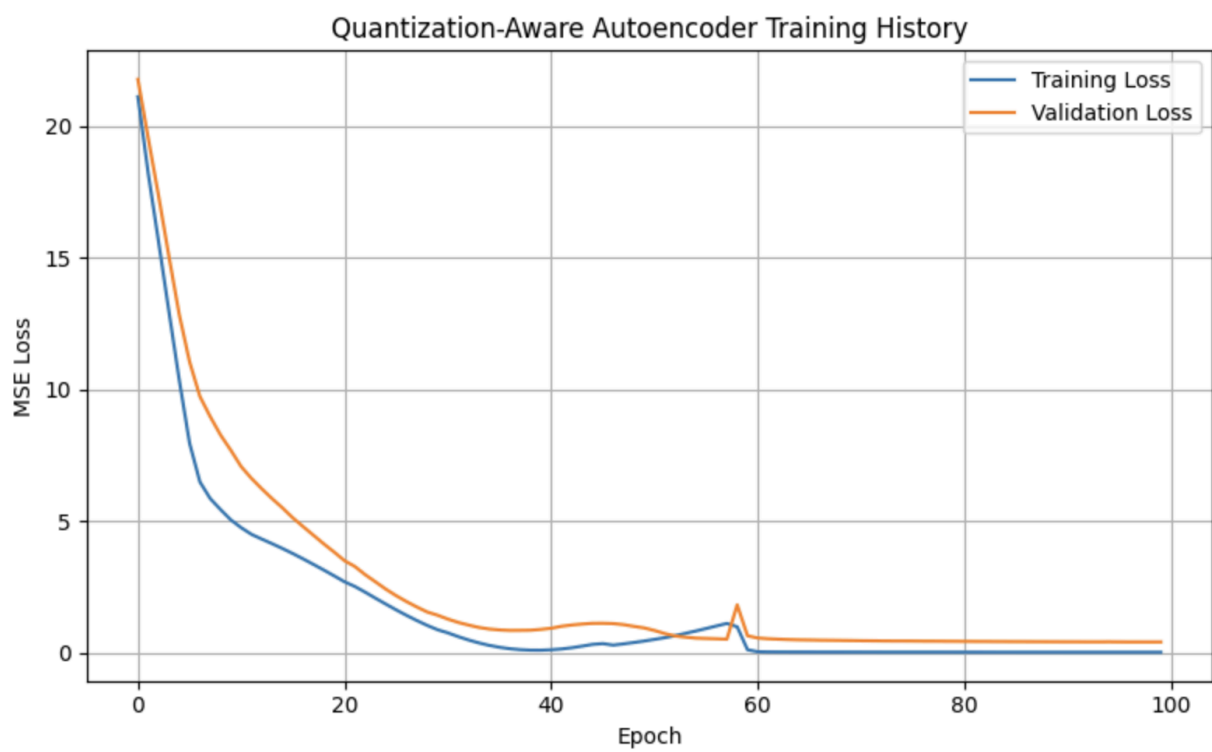
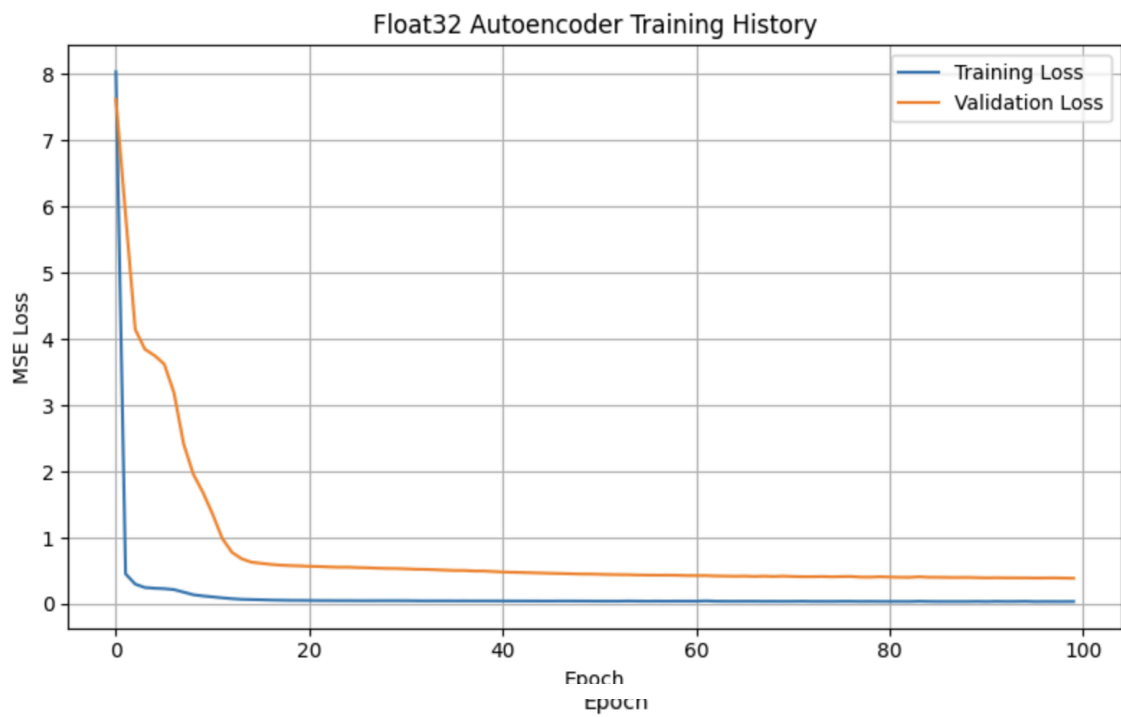
Part 1:

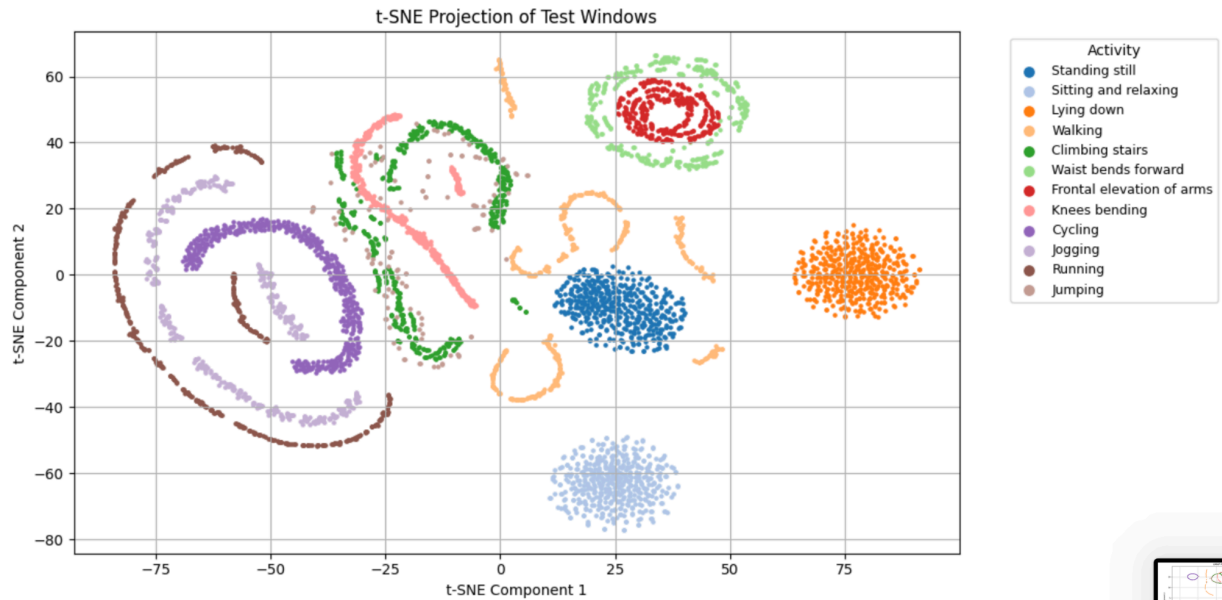
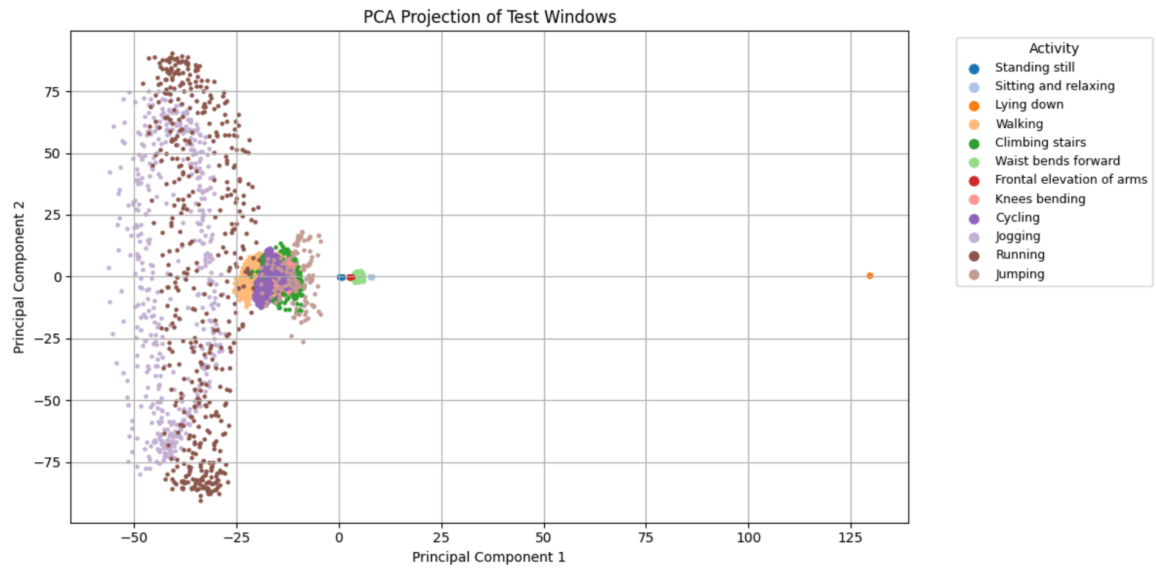
1. The raw inputs are accX, accZ, and accY. These are accelerometer readings along their respective axis
2. The features passed into the NN classifier are the spectral features, and there are 33 input features. The names are RMS, Peak 1 Height, Spectral Power 2.0-5.0, Peak 1 Freq, and Peak 2 Freq.
3. The NN classifier produces a probability distribution over 2 output classes: nominal and off
4. The K-means anomaly detector receives a subset of the same spectral features generated by the DSP block: accX RMS, accX Peak 1 Height, accY Spectral Power 2.0-5.0 Hz, accZ Spectral Power 2.0-5.0 Hz

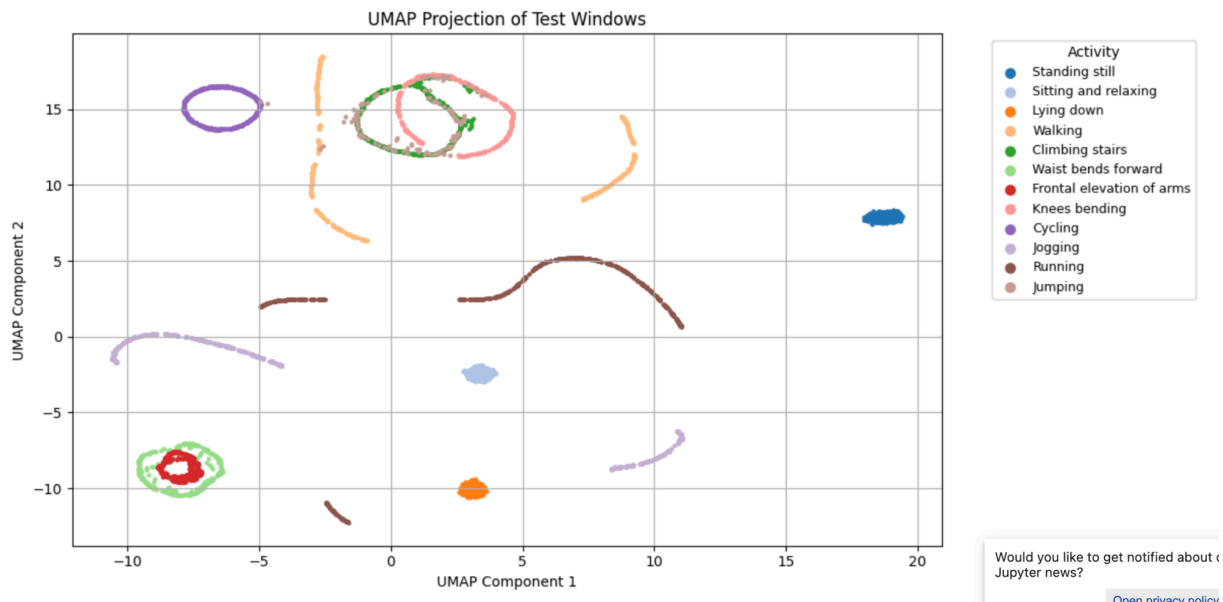
```
anomaly_edgeimpulse — node ~/.nvm/versions/node/v24.13.0/bin/edge-i...
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 593 us, anomaly 536 us, postprocessing 102 us
#Classification predictions:
  nominal: 0.234375
  off: 0.765625
Anomaly prediction: -0.114336
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 594 us, anomaly 538 us, postprocessing 49 us
#Classification predictions:
```

- 5.
6. When the Nano 33 BLE Sense was held near the fan in its normal running state, the classifier consistently reported nominal with high confidence and the anomaly score remained low, indicating that the input feature vector sat close to one of the learned K-means cluster centroids. Switching the fan off shifted the predicted class to off as expected, again with a low anomaly score.

Part 2

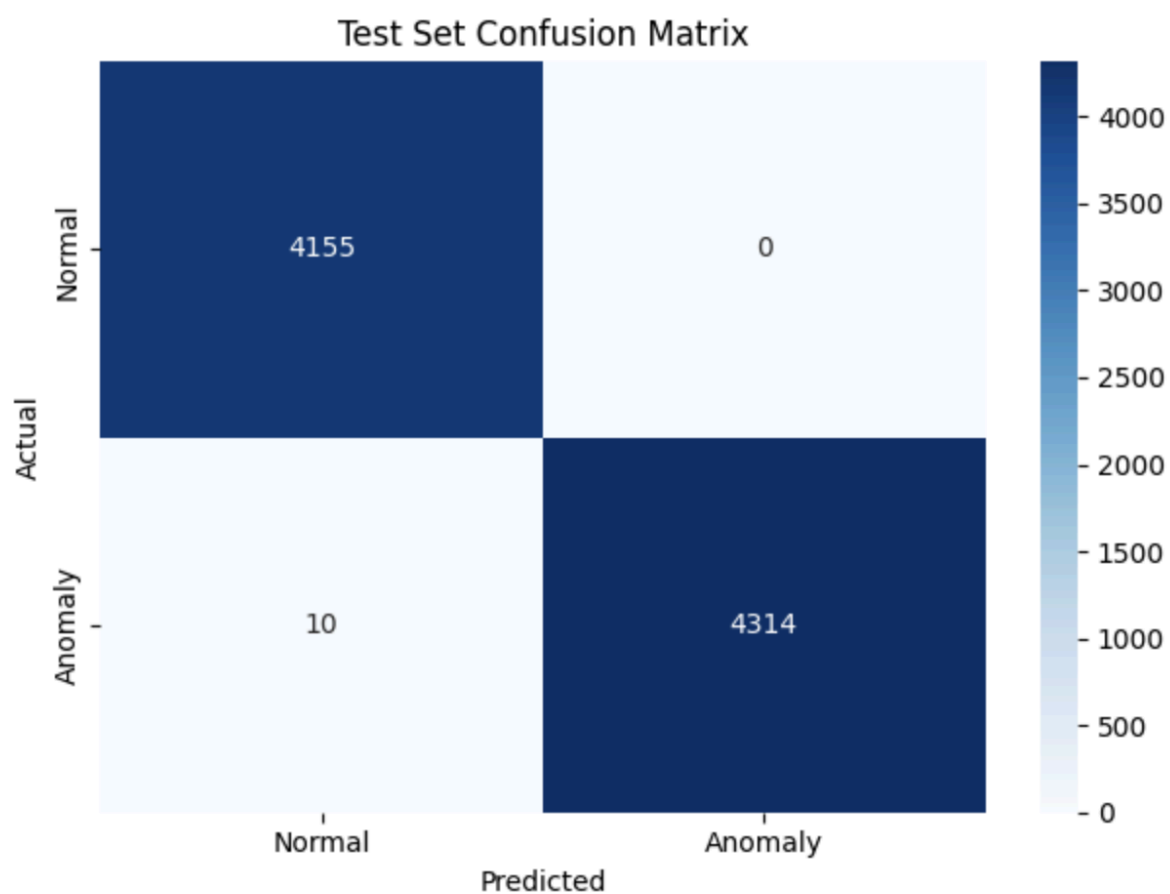
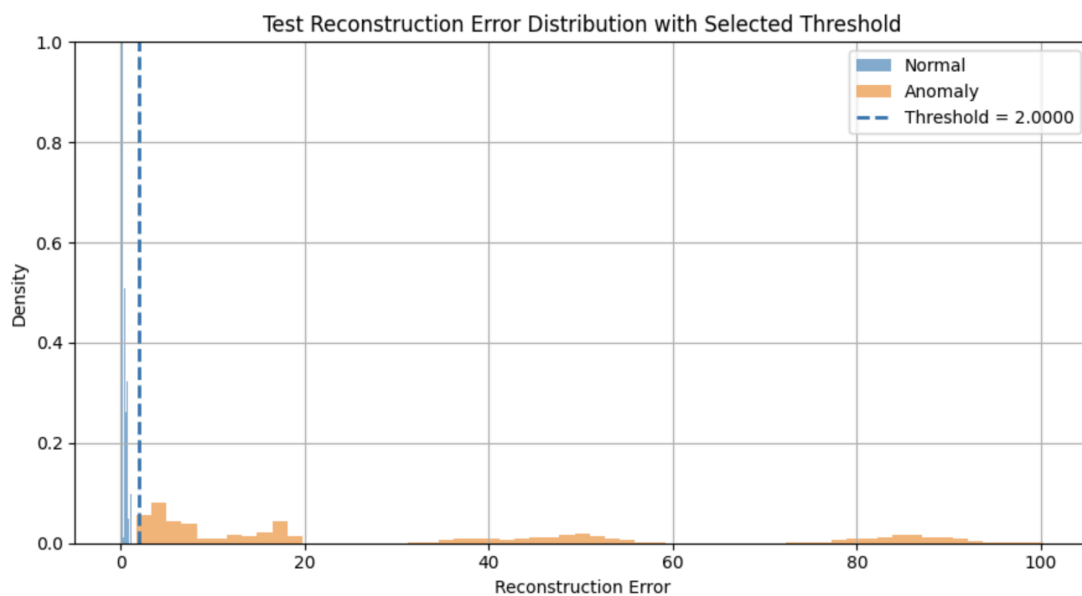






Test anomaly mean=32.081051, stderr=0.465342, min=1.654439, max=100.240952





int8 created tensorflow Lite ANNker delegate for CPU

```
Train set evaluation: int8 model
              precision    recall  f1-score   support

   Normal (0)         0.99      1.00      1.00     10478
  Anomaly (1)         1.00      0.99      1.00     10872

       accuracy              1.00     21350
      macro avg         1.00      1.00      1.00     21350
     weighted avg         1.00      1.00      1.00     21350

Test set evaluation: int8 model
              precision    recall  f1-score   support

   Normal (0)         1.00      1.00      1.00      4155
  Anomaly (1)         1.00      1.00      1.00      4324

       accuracy              1.00      8479
      macro avg         1.00      1.00      1.00      8479
     weighted avg         1.00      1.00      1.00      8479
```

```
Result: Normal activity
Reconstruction error: 1.266466
Result: Normal activity
Reconstruction error: 1.264020
Result: Normal activity
Reconstruction error: 1.222025
Result: Normal activity
Reconstruction error: 1.218426
Result: Normal activity
Reconstruction error: 1.227132
Result: Normal activity
Reconstruction error: 1.226697
Result: Normal activity
Reconstruction error: 1.225715
```

Part 3:

What threshold did you choose for anomaly detection, and why?

I chose a reconstruction-error threshold of 2.0 because the histogram of test-set reconstruction errors showed a wide valley between the normal and anomaly distributions, and 2.0 sat squarely in the middle of that gap

How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder is trained only on normal data, so it learns a compressed representation that reconstructs normal windows with low error but fails to reconstruct unfamiliar inputs accurately. Anomalous activity therefore produces a noticeably higher mean squared error between input and reconstruction, turning a scalar error value into a simple threshold-based decision.

What information from the Python notebook must be preserved when deploying the model to Arduino?

The int8 TFLite model itself (embedded as `g_model` in `autoencoder_model.cc`) and its quantization parameters (`input_scale`, `input_zero_point`, `output_scale`, `output_zero_point`) must be preserved so the Arduino can correctly convert between float sensor readings and int8 tensor values. The input shape (100 samples $\times$ 3 axes), axis order, MSE reconstruction-error formula, and the chosen threshold (2.0) must also match.

Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

A neural network only behaves correctly on inputs that match its training distribution, so any mismatch in sampling rate, axis order, window length, or quantization formula will make normal activity look anomalous and produce false positives. This is especially critical for quantized models, where the int8 representation embeds an assumed input range that must be applied identically on the device.

What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The threshold is fixed at compile time, so the device cannot adapt to sensor drift or changes in operating environment, and each window is evaluated independently without temporal smoothing, so a single noisy window can trigger a spurious alert.